SYSTEM ANALYSIS & TECHNICAL IMPLEMENTATION BLUEPRINT

DAFCustomer Support and Monitoring System

Detailed Functional Specifications, Role-Based Access Control & Architecture for Engineering

1. Executive Summary & System Purpose

The DAFCustomer Support and Monitoring System transitions DAF Tech Engineering from fragmented manual sheets into a standardized, audited, and role-governed customer feedback platform. The system enforces proactive weekly outreach to client organizations and their specific software modules, records structured customer satisfaction indicators, and computes objective daily and weekly staff performance metrics.

2. Detailed Role & Permission Matrix

The system implements fine-grained dynamic claims-based authorization on top of ASP.NET Core Identity. Below is the operational role model specifying required actions and system touchpoints:

Role	Core Responsibilities	Granular System Permissions
Support Agent Operational	Direct outreach to assigned customer contacts; capturing call logs, ratings, and issue notes; tracking personal daily quota.	`Schedule.ViewOwn`, `Feedback.Create`, `Feedback.ViewOwn`, `Performance.ViewOwn`, `Organization.ViewAssigned`
Team Lead Supervisory	Generating schedules; balancing agent assignments; monitoring team performance and unresolved blockers; reviewing overdue calls.	All Agent permissions + `Schedule.Generate`, `Schedule.Publish`, `Assignment.Manage`, `Feedback.ViewTeam`, `Performance.ViewTeam`
Director Executive	High-level strategic visibility; reviewing company-wide SLA completion, customer satisfaction trends, and problematic modules.	`Dashboard.ViewExecutive`, `Report.ExportAll`, `Performance.ViewAll`, `Audit.ViewSummaries` (Read-Only across all entities)
System Administrator Governance	Master data management; user lifecycle; configuring roles and permission mappings; audit oversight and system parameters.	Full CRUD: `User.Manage`, `Role.Manage`, `Organization.Manage`, `Module.Manage`, `Contact.Manage`, `System.Configure`

3. End-to-End Functional Workflows

3.1 Weekly Schedule Generation & Distribution

1. **Assignment Resolution:** The backend queries active `SupportAssignment` mappings linking an agent to organizations and optional modules.
2. **Contact Person Association:** When a module is specified, the system prioritizes module-specific active contacts; otherwise, it falls back to the organization's primary contact.
3. **Batch Generation:** The Team Lead runs the schedule generation service for the upcoming cycle (Monday–Sunday), generating draft `ScheduleItem` records.
4. **Publishing:** Once finalized and published, items lock into the agent's active queue and become eligible for performance evaluation calculations.

3.2 Feedback Capture & Validation Lifecycle

1. **Pre-Population:** When opening a feedback form from a scheduled contact, the caller identity is strictly bound to the authenticated JWT claims (`CallerUserId`). Organization, contact person, and phone number are auto-filled.
2. **Input Capture:** Mandatory rating (1–5 scale), priority classification (*Low, Medium, High, Blocker*), status progression (*Not Resolved, Assigned, On Progress, Resolved*), and remarks text.
3. **State Transition:** Saving a valid feedback entry updates the corresponding `ScheduleItem.Status` from `Pending` to `Completed` and sets `CompletedAt = UTC_NOW`. Historical records remain immutable for auditing purposes.

> **Core Performance Metric Rule:** Performance (%) is calculated dynamically as `(Completed Contacts / Planned Contacts) × 100` over any selected time window (daily or weekly). Only contacts within *Published* schedules are included in denominator calculations.

4. Technical Implementation Specifications

- **Backend Architecture (.NET 9):** Structured into Clean Architecture layers (`DAFCustomerSupport.Api` , `Application` , `Domain` , `Infrastructure`). Domain entities enforce encapsulation, while Application commands and queries use FluentValidation.
- **Database & Data Model (SQL Server / EF Core 9):** Master tables (`Organizations` , `Modules` , `ContactPersons`) with soft-delete support (`IsActive`). High-frequency schedule queries utilize indexed foreign keys on `WeeklyScheduleId` , `AssignedUserId` , and `PlannedDate` .
- **Frontend Architecture (Angular 18+):** Feature module layout (`auth` , `dashboard` , `schedules` , `feedback` , `admin`). HTTP interceptor handles automatic JWT attachment and refresh token rotation, while Angular Route Guards enforce permissions before component rendering.